

Java CI/CD Pipeline

THE COMPLETE GUIDE

From Zero Knowledge to Interview-Ready
Foundations - Tools - Pipeline - Interview Prep

TECHNOLOGY STACK

Java - Maven - Jenkins - SonarQube - Nexus
Docker - Kubernetes - Helm - AWS - Git

DEEP COMPARISON: JAVA vs PYTHON vs NODE.JS

70+ Interview Questions - Commands Cheat Sheet - Architecture Diagrams

Table of Contents

1	Foundation: Programming Languages, Compilation, and Why Java Exists	4
2	The Java Ecosystem: JDK, JRE, JVM, and Bytecode	7
3	Maven: The Build Tool That Runs Java Projects	9
4	Java Web Applications: Servlets, WAR Files, and Tomcat	12
5	CI/CD Concepts: What and Why	14
6	The Tools in a Java CI/CD Pipeline	16
7	The Complete Pipeline: Stage by Stage	20
8	Building It: Infrastructure and Hands-On Setup	25
9	Jenkins Shared Libraries (Enterprise Pattern)	28
10	Java vs Python vs Node.js: Deep Comparison	30
11	Interview Preparation: 70+ Questions with Answers	34
12	Commands Cheat Sheet	40
13	Glossary: Every Term Defined	42

HOW TO USE THIS GUIDE: This document assumes zero prior knowledge of Java, CI/CD, or any of the tools involved. Every concept is explained from first principles. Read sequentially, or jump to specific sections via the table of contents.

Foundation: Programming Languages, Compilation, and Why Java Exists

Start here. Skip nothing.

Why this section exists

Before you can understand a Java CI/CD pipeline, you must understand WHY Java is built the way it is. The entire pipeline architecture — every stage, every tool, every command — exists because of decisions made when Java was designed. Without this foundation, you will memorize commands without understanding why they exist. With this foundation, the pipeline becomes obvious.

1.1 The Two Types of Programming Languages

Every programming language falls into one of two camps based on how the computer eventually runs the code:

Type	How it works	Examples	Speed	Error catching
Compiled	Source code -> Compiler -> Machine code -> CPU runs it directly	C, C++, Go, Rust	Very fast	At compile time (before running)
Interpreted	Source code -> Interpreter reads each line and executes it	Python, JavaScript, Ruby	Slower	At runtime (while running)
Hybrid (bytecode)	Source -> Compiler -> Bytecode -> Virtual Machine executes bytecode	Java, C#, Kotlin, Scala	Fast	At compile time AND runtime

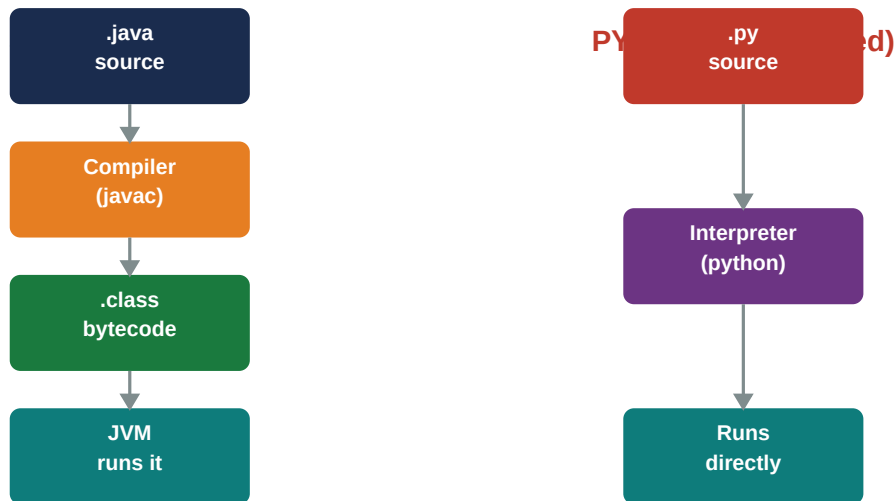
Java is in the third category — a hybrid. This single fact is the root of everything that follows.

1.2 Why Java Was Designed This Way

In the 1990s, Sun Microsystems wanted a language that could solve a real problem: software written on a Windows machine could not run on a Mac, Linux, or any other operating system. Each platform required recompiling the code with a platform-specific compiler. This was painful.

Java's answer was the famous slogan: **"Write Once, Run Anywhere"**. The solution was elegant:

1. Compile Java source code into an intermediate format called **bytecode**
2. Create a small program — the Java Virtual Machine (JVM) — that runs bytecode
3. Build a JVM for every operating system
4. Now bytecode runs anywhere a JVM exists, without recompilation



KEY INSIGHT: Bytecode is the secret. It is platform-independent code that the JVM translates into machine code at the moment of execution. This is why a Java .jar file built on Windows runs unchanged on Linux EC2 servers. Without bytecode, there would be no Java ecosystem and no Maven and no Tomcat.

1.3 The Consequences of Java Being Compiled

Java's compilation step has massive downstream effects that shape the entire pipeline:

Consequence	Why it matters
You need a build step	Java cannot run from .java source files. You must compile first. This is why "mvn compile" exists.
You produce binary artifacts	The output is .class, .jar, or .war — binary files. You ship these, not source code.
Strict typing catches errors early	If you write "int x = "hello";" the compiler refuses. Python would only crash when that line runs.
Need package management	Compiled code references compiled libraries. You need a tool (Maven) to manage these dependencies.
Artifact repositories matter	Built JARs/WARs need permanent storage. Hence Nexus, Artifactory, Maven Central exist.
Multi-stage Docker builds	Dockerfile needs Maven during build but only JRE at runtime. Hence multi-stage builds.

When you understand this, every later decision in the pipeline makes sense. We build because Java must be compiled. We use Maven because compiled code needs dependency management. We use Nexus because binary artifacts need versioned storage. We use Tomcat because Java web apps need a JVM environment to run in. Everything connects.

The Java Ecosystem: JDK, JRE, JVM, and Bytecode

The three letters everyone confuses

2.1 The Critical Distinction

New developers constantly confuse JDK, JRE, and JVM. This is the #1 cause of "I installed Java but my code won't compile" problems. Memorize this diagram — it will be on every Java interview:



Component	Stands for	Contains	When you need it
JVM	Java Virtual Machine	Just the runtime that executes bytecode	Theoretical — never installed alone
JRE	Java Runtime Environment	JVM + standard libraries (java.lang, java.util, etc.)	When you only need to RUN Java apps (e.g., end users)
JDK	Java Development Kit	JRE + javac compiler + dev tools (jar, javadoc, debugger)	When you need to BUILD Java apps (developers, CI servers)

PRACTICAL RULE: Always install JDK on your build servers and developer machines. JRE alone cannot compile code. In our pipeline, EC2 needs JDK (not JRE) because Jenkins runs Maven which runs javac.

2.2 Java Version Numbering

Java versions are confusing because of historical naming changes:

Java 1.0 (1996), 1.1, 1.2, 1.3, 1.4, 1.5 -> rebranded as "Java 5" (2004)
 Java 6, 7, 8 (2014) -> the most legacy-used version
 Java 9, 10, ... 11 (LTS), 17 (LTS), 21 (LTS) -> current era

LTS = Long Term Support. Oracle releases LTS versions every 2-3 years and supports them for many years. Non-LTS versions (12, 13, 14, 15, 16, 18, 19, 20) are short-lived and not used in production. Enterprises run LTS: Java 8 (legacy), Java 11, Java 17 (current sweet spot), or Java 21 (newest).

For our pipeline, we use Java 17 — current LTS, widely supported by Maven, Tomcat 9+, and SonarQube.

2.3 Bytecode in Practice

When you run "javac HelloWorld.java", you get "HelloWorld.class". Let's look at what happens:

```
# Source code (HelloWorld.java) public class HelloWorld { public static void main(String[]  
args) { System.out.println("Hello"); } } # After: javac HelloWorld.java # You get:  
HelloWorld.class (binary file, not human readable) # Inspect bytecode with: javap -c HelloWorld  
0: getstatic #2 // Field java/lang/System.out 3: ldc #3 // String Hello 5: invokevirtual #4 //  
Method println 8: return
```

This bytecode is identical on every operating system. The JVM on Windows reads it and produces Windows machine instructions. The JVM on Linux reads the same bytes and produces Linux instructions. Same input, different output, same program behavior. This is platform independence in action.

Maven: The Build Tool That Runs Java Projects

The most important tool to understand

3.1 What Maven Solves

Before Maven (2004), building a Java project meant manually:

- Downloading dependency JARs from various websites
- Putting them in correct folders
- Running javac with the right classpath
- Running tests with another command
- Manually zipping outputs into JAR/WAR files
- Different projects had completely different build scripts

This was painful and inconsistent. Maven introduced two ideas that solved everything:

1. **Convention over Configuration:** follow a standard folder structure, Maven needs zero config to find your code
2. **Dependency Management:** declare what your project needs in pom.xml — Maven downloads it automatically

3.2 Maven Standard Folder Structure

Every Maven project on Earth looks like this. You did not invent this structure — Maven enforces it:

```
my-project/ |-- pom.xml <- the project configuration |-- src/ | |-- main/ | | |-- java/ <-  
production Java source code | | |-- `-- com/company/App.java | | |-- resources/ <- config files,  
properties | | |-- `-- webapp/ <- web files (for WAR projects) | | |-- `-- WEB-INF/ | |-- test/ | |--  
java/ <- test code (JUnit tests) | |-- `-- com/company/AppTest.java |-- target/ <- build output  
(auto-created) |-- classes/ <- compiled .class files |-- my-project-1.0.0.war <- the final  
artifact
```

IMPORTANT: The package path in Java (com.company) MUST match the folder structure (com/company/). This is a JVM requirement, not just a Maven convention. Maven enforces it.

3.3 The pom.xml — Project Object Model

pom.xml is the heart of Maven. It defines what your project IS, what it depends on, and how to build it. Every interview will ask "what is pom.xml?". Know this file inside out.

```
4.0.0 com.techcorp payment-service 1.0.0 war 17 17 17 javax.servlet javax.servlet-api 4.0.1  
provided junit junit 4.13.2 test nexus-releases http://nexus:8081/repository/maven-releases/
```

3.4 Maven Coordinates (groupId, artifactId, version)

Every Java library in the world has three coordinates that uniquely identify it. Think of them as address: groupId is the city, artifactId is the street, version is the house number.

Coordinate	Meaning	Example
groupId	Organization namespace, like a reverse domain name	com.techcorp.payments
artifactId	The specific project name within that org	payment-service
version	The exact version of this artifact	1.0.5

When Maven sees "junit:junit:4.13.2" in pom.xml, it looks up this exact artifact in Maven Central, downloads it, and adds it to your classpath. No manual downloading. No version confusion. No "it works on my machine".

3.5 Dependency Scopes

Each dependency has a scope. This is critical for interviews:

Scope	Behavior	Example use case
compile (default)	Needed to compile, included in final artifact	Application logic libraries (Apache Commons, etc.)
provided	Needed to compile, NOT included in final artifact (runtime provides it)	javax.servlet-api (Tomcat provides this)
runtime	Not needed to compile, only at runtime	JDBC drivers (MySQL connector)
test	Only needed for tests, not in final artifact	JUnit, Mockito
system	Like provided, but you specify the JAR location manually	Rare, avoided in modern projects

INTERVIEW GOTCHA: The "provided" scope is one of the most common interview questions. If you forget it, the servlet-api.jar gets bundled inside your WAR. Tomcat already has it. Two copies cause ClassLoader conflicts and runtime errors.

3.6 The Maven Build Lifecycle

Maven builds happen in a sequence of "phases". Each phase includes everything before it. This is THE concept to understand for interviews:



Phase	What happens	Output
validate	Check pom.xml is correct, all required info present	No output
compile	javac compiles .java -> .class	target/classes/
test	Run JUnit/TestNG tests	target/surefire-reports/
package	Bundle compiled code + resources into WAR/JAR	target/app-1.0.0.war
verify	Run integration tests, additional checks	Reports
install	Copy artifact to LOCAL Maven repo (~/.m2/repository/)	Local cache
deploy	Upload artifact to REMOTE repo (Nexus, Artifactory)	Artifact in Nexus

Key rule: when you run "mvn deploy", Maven automatically runs every previous phase first: validate -> compile -> test -> package -> verify -> install -> deploy. You cannot skip phases.

The most common commands you will actually use:

```
mvn clean # delete target/ folder mvn clean compile # compile only mvn clean test # compile +  
run tests mvn clean package # compile + test + create artifact mvn clean install # everything +  
copy to local repo mvn clean deploy # everything + push to Nexus
```

Java Web Applications: Servlets, WAR Files, and Tomcat

How Java handles web requests

4.1 What is a Java Web Application

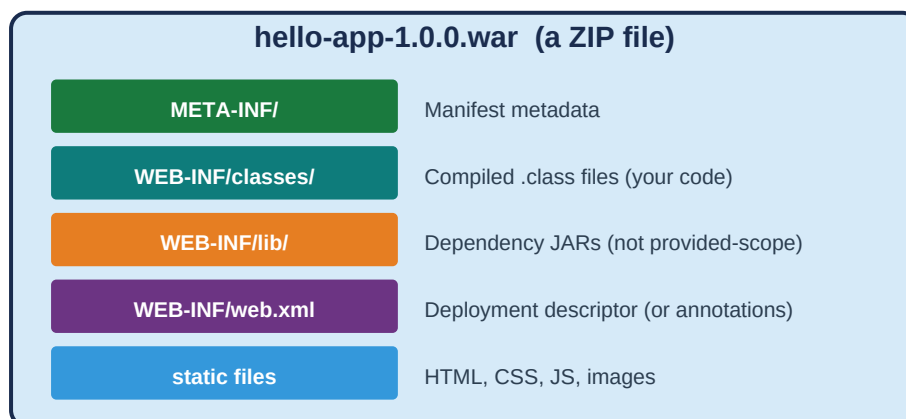
A web application receives HTTP requests and sends HTTP responses. In Java, this is built on top of the Servlet API — a standard interface defined since 1997. A "servlet" is a Java class that handles HTTP requests.

```
package com.techcorp; import javax.servlet.http.*; import javax.servlet.annotation.WebServlet;
@WebServlet("/hello") public class HelloServlet extends HttpServlet { protected void
doGet(HttpServletRequest req, HttpServletResponse resp) { resp.getWriter().println("Hello!"); }
}
```

This Java class becomes a web endpoint. When a browser requests "/hello", this code runs.

4.2 The WAR File - What's Inside

WAR = Web Application aRchive. It is a ZIP file with a specific structure. When you run "mvn package" on a Java web project, this is what gets created:



REMEMBER: Critical: a WAR contains COMPILED .class files, not .java source. Source code never leaves your laptop. Compiled bytecode is what ships. This is true for every compiled language.

4.3 JAR vs WAR vs EAR

Three Java archive types — each interview will ask the difference:

Archive	Stands for	Use case	Runs how
.jar	Java ARchive	Standalone Java apps, libraries	java -jar app.jar (executable JARs)
.war	Web ARchive	Web applications (servlets, JSPs)	Deployed inside a servlet container like Tomcat
.ear	Enterprise ARchive	Enterprise Java apps (rare in 2025)	Deployed in full Jakarta EE servers like WildFly

In modern Spring Boot projects, web apps are packaged as JARs (not WARs) because Spring Boot embeds Tomcat directly. The JAR contains both your code AND Tomcat itself. Run with "java -jar app.jar" and your web app starts. No external Tomcat needed.

4.4 What is Tomcat

Tomcat is a **servlet container** — a runtime environment that knows how to execute Java web apps. It is itself a Java application that:

- Listens on a TCP port (default 8080)
- Accepts HTTP requests
- Reads .war files in its webapps/ folder
- Loads servlet classes from those WARs
- Routes incoming requests to the correct servlet
- Manages threads, sessions, and lifecycle
- Sends responses back

Without Tomcat (or another servlet container like Jetty, WildFly, GlassFish), your .war file is inert. It cannot listen on a port. It cannot receive HTTP requests. Tomcat provides all of that.

The deployment process is literally a file copy:

```
# Step 1: Copy your WAR to Tomcat's webapps folder sudo cp hello-app-1.0.0.war /opt/tomcat9/webapps/ # Step 2: Tomcat automatically detects the new WAR # - Extracts it into a folder of the same name # - Loads servlet classes # - Routes traffic to /hello-app-1.0.0/... # That's it. No restart needed. This is "hot deployment".
```

5.1 What is CI/CD

CI/CD stands for **Continuous Integration / Continuous Delivery** (or Deployment). It is a software engineering practice where every code change automatically goes through build, test, security scan, and deployment without manual intervention.

Term	Definition	What automation includes
CI (Continuous Integration)	Every push triggers automated build and tests	Build, unit tests, code quality, security scans
CD (Continuous Delivery)	Every successful build is auto-prepared for deployment, requires manual approval to deploy	CI + artifact storage + staging deployment
CD (Continuous Deployment)	Every successful build automatically deploys to production	CI + Delivery + auto-production deployment

In most large enterprises, the standard is CI + Continuous Delivery (with manual approval for production). Full Continuous Deployment is used by tech companies (Netflix, Google) but rare in finance, healthcare, banking due to compliance requirements.

5.2 Why CI/CD Exists - The Pain It Solves

Before CI/CD, software development looked like this:

The bad old days:

- Developer writes code on their laptop for 2 weeks
- "It works on my machine"
- Tries to merge with team's code -> 500 conflicts
- Manual build process takes 4 hours, often fails
- QA finds bugs 3 weeks later because integration was delayed
- Deployment is a Friday night ritual involving 6 people
- Production breaks Sunday morning, nobody knows why
- Rollback is a manual nightmare

With CI/CD:

- Every code change is integrated within minutes
- Automated tests catch bugs immediately
- Build process is identical every time
- Security scans run on every change
- Deployments happen automatically, predictably
- Rollback is one command
- Bugs are caught in minutes, not weeks

5.3 Pipeline as Code

A modern CI/CD pipeline is defined by a file called **Jenkinsfile** (for Jenkins) or **.gitlab-ci.yml** (for GitLab CI). This file lives in your code repository alongside the application code.

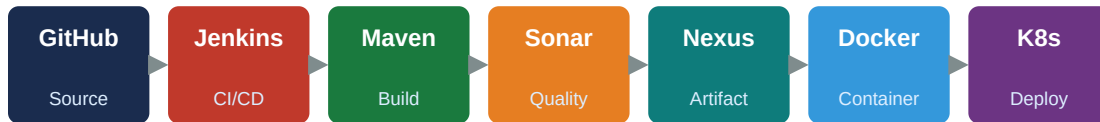
Why? Three reasons:

1. **Version controlled:** pipeline changes go through code review like application code
2. **Reproducible:** the pipeline is identical for every developer and every environment
3. **Auditable:** any pipeline change has a Git commit history showing who, what, when, why

The alternative — configuring the pipeline through a UI — is fragile, untracked, and impossible to audit. No serious company uses UI-based pipelines anymore.

The Tools in a Java CI/CD Pipeline

Every tool, what it does, why it exists



GitHub (Source Control)

What it is: A web-based platform for storing Git repositories. The single source of truth for your code.

Where: cloud-hosted at github.com (or self-hosted GitHub Enterprise)

How developers use it: clone, branch, commit, push, pull request, merge

Triggers the pipeline: when code is pushed, GitHub sends a webhook to Jenkins

Jenkins (CI/CD Orchestrator)

What it is: An automation server that runs pipeline stages. The "brain" of CI/CD.

Where: installed on a server (EC2), runs as a systemctl service, port 8080

How it works: reads Jenkinsfile from your Git repo, executes stages on build agents

What it does: clones code, runs Maven, calls SonarQube, pushes to Nexus, deploys to Tomcat/K8s

Maven (Build Tool)

What it is: The Java build automation tool. Compiles code, manages dependencies, packages artifacts.

Where: installed on Jenkins agents (configured in Jenkins Global Tools)

How: Jenkins runs "mvn clean package" on the agent — same command developers run locally

Output: produces .war or .jar file in the target/ folder

JUnit (Testing Framework)

What it is: The standard Java unit testing framework. Runs during "mvn test" phase.

Where: a dependency in pom.xml with scope=test

Test files: live in src/test/java/, named *Test.java (Maven Surefire convention)

Output: XML reports in target/surefire-reports/ that Jenkins reads and visualizes

SonarQube (SAST)

What it is: Static Application Security Testing. Analyzes source code for bugs, vulnerabilities, code smells.

Where: separate server (EC2) with PostgreSQL database, port 9000
How: Maven plugin "mvn sonar:sonar" sends code analysis to SonarQube server
Quality Gate: pass/fail policy — pipeline stops if Quality Gate fails

OWASP Dependency Check / NexusIQ (SCA)

What it is: Software Composition Analysis. Scans third-party libraries for known security vulnerabilities (CVEs).

Where: OWASP Dependency Check runs in Maven; NexusIQ is a separate Sonatype product
What it checks: every JAR your project depends on, against the NVD CVE database
Why: your code might be safe, but log4j 2.14 is not. SCA catches this.

Nexus (Artifact Repository)

What it is: A storage server for built artifacts. Stores .war/.jar files with version history.

Where: separate server (EC2), port 8081
Repositories: maven-releases (immutable), maven-snapshots (mutable), maven-public (proxy cache)
How: "mvn deploy" uploads to Nexus; teams pull artifacts from Nexus URLs

Docker (Containerization)

What it is: Packages your app + JVM + dependencies + OS libraries into a portable container image.

Where: Docker daemon installed on Jenkins agent
How: Dockerfile in repo describes how to build the image; "docker build" creates it
Output: a versioned Docker image (myapp:1.0.5) pushed to a container registry

Container Registry

What it is: Storage for Docker images, like Nexus but for container images.

Options: DockerHub (public), AWS ECR, Google GCR, Harbor (self-hosted), Nexus (also supports)
How: "docker push myrepo/myapp:1.0.5" uploads; "docker pull" downloads
Tagged versions: production uses specific tags like 1.0.5, never "latest"

Kubernetes (Container Orchestration)

What it is: Runs containers across multiple servers, handles scaling, networking, failure recovery.

Where: a cluster of nodes (EC2 instances, or AWS EKS managed service)
Components: Pods (containers), Deployments (manage pods), Services (networking)
How: Helm or kubectl applies YAML manifests; K8s pulls images and runs pods

Helm (Kubernetes Package Manager)

What it is: Templates Kubernetes YAML files into reusable, parameterized "charts".

Where: command-line tool that talks to a Kubernetes cluster

Why: instead of writing 20 raw YAML files per service, use one parameterized chart

Commands: helm install, helm upgrade, helm rollback

AWS (Infrastructure)

What it is: Cloud provider for the underlying compute, storage, and networking.

Services used in this pipeline:

- EC2: virtual servers for Jenkins, SonarQube, Nexus, Tomcat
- S3: object storage for artifact backups, logs, static assets
- ELB/ALB: load balancer distributing traffic to multiple instances
- ASG: auto-scales EC2 instances based on demand

The Complete Pipeline: Stage by Stage

What happens, in what order, and why

This section walks through every stage of the pipeline from "developer pushes code" to "app runs in production". Read this multiple times. Every interview about CI/CD comes back to this flow.

Stage 1: Developer Pushes Code	Trigger
Action: Developer push to GitHub	
Developer commits code to a feature branch and pushes to GitHub. They raise a Pull Request to merge into "develop" branch (Gitflow pattern). GitHub fires a webhook to Jenkins at http://jenkins/github-webhook/ . Jenkins receives the webhook payload (commit SHA, branch name, repository URL) and adds a build to the queue.	
<i>Output: a new Jenkins build is queued. Nothing has compiled yet.</i>	
Stage 2: Jenkins Assigns Build to Agent	Orchestration
Action: Jenkins controller -> available agent	
The Jenkins controller (master) does not run builds directly — it orchestrates. It finds an available build agent (a worker EC2 server or Kubernetes pod) and assigns the build. The agent has Java, Maven, Docker pre-installed.	
<i>Output: build starts running on a specific agent. Build #47 is now in progress.</i>	
Stage 3: Code Checkout	Checkout
Action: git clone the repository	
The agent runs "git clone" or "git fetch" to pull the latest code into a workspace folder (/var/lib/jenkins/workspace/). This is the Jenkinsfile "checkout scm" step. Every build starts with a fresh code clone — this is intentional, it prevents stale-state bugs.	
<i>Output: full source code is now on the agent at /var/lib/jenkins/workspace/payment-service/</i>	
Stage 4: Maven Build	Build
Action: mvn clean package	
The agent runs "mvn clean package -DskipTests". Maven: - Deletes the previous target/ folder (clean) - Downloads any new dependencies from Nexus/Maven Central - Compiles all .java files into .class bytecode - Bundles compiled code + resources + dependencies into target/payment-service-1.0.5.war If compilation fails, the pipeline stops here. No further stages run.	
<i>Output: target/payment-service-1.0.5.war exists on the agent disk</i>	
Stage 5: Unit Tests	Test

Action: mvn test	
JUnit tests run via Maven Surefire plugin. Every method annotated with @Test is executed. Results are written to target/surefire-reports/ as XML files. Jenkins reads these XML files and displays pass/fail counts in the build UI. If ANY test fails, the pipeline can be configured to stop or continue. Enterprise rule: tests must pass. Failed tests block the pipeline.	
<i>Output: test results visible in Jenkins UI. Code coverage report (JaCoCo).</i>	
Stage 6: SonarQube Analysis (SAST)	Code Quality
Action: mvn sonar:sonar	
The Maven SonarQube plugin sends source code, compiled bytecode, and coverage reports to the SonarQube server. SonarQube analyzes the code and detects: - Bugs (null pointer dereferences, resource leaks) - Vulnerabilities (SQL injection, hardcoded passwords) - Code smells (overly complex methods, duplications) - Coverage gaps (untested code) Results are stored in SonarQube database, visible at http://sonar:9000/	
<i>Output: analysis report posted to SonarQube; Jenkins gets back a "Quality Gate" pass/fail</i>	
Stage 7: Quality Gate	Policy
Action: waitForQualityGate abortPipeline: true	
Jenkins waits for SonarQube to compute the Quality Gate result. A Quality Gate is a pass/fail policy configured in SonarQube, for example: "no new bugs, no new vulnerabilities, coverage >= 80%, duplications < 3%". If the Quality Gate FAILS, Jenkins aborts the pipeline. This is the security/quality enforcement point. Bad code never proceeds past here.	
<i>Output: pipeline either continues or aborts based on Quality Gate</i>	
Stage 8: OWASP Dependency Check (SCA)	Security
Action: mvn dependency-check:check	
OWASP Dependency Check examines every JAR your project depends on (transitively too — dependencies of dependencies). It compares each JAR against the National Vulnerability Database (NVD) and flags any with known CVEs. Configuration: -DfailBuildOnCVSS=7 means fail the build if ANY dependency has a CVE with CVSS score >= 7 (High). This catches things like Log4Shell (CVE-2021-44228) before they reach production.	
<i>Output: dependency-check-report.html with findings; build fails if critical CVEs found</i>	
Stage 9: Push Artifact to Nexus	Storage
Action: mvn deploy	

Maven uploads the built .war to the Nexus repository specified in pom.xml distributionManagement. The artifact is stored with its version number permanently — every version ever built is retrievable.
URL format: `http://nexus:8081/repository/maven-releases/com/techcorp/payment-service/1.0.5/payment-service-1.0.5.war`
This enables rollback: if 1.0.5 breaks production, you redeploy 1.0.4 from Nexus instantly.

Output: artifact stored in Nexus with checksums (.md5, .sha1) for integrity verification

Stage 10: Docker Image Build

Containerization

Action: docker build

Jenkins runs `"docker build -t payment-service:47 ."` which reads the Dockerfile in the repo. The Dockerfile typically uses multi-stage builds:

Stage 1 (builder): use a Maven image, compile the code

Stage 2 (runtime): use a JRE-only image, copy ONLY the compiled JAR/WAR from stage 1

Result: a small (~200MB) image with just the JRE and your app, not the build tools.

Output: Docker image "payment-service:47" exists in the local Docker daemon

Stage 11: Push Docker Image to Registry

Distribution

Action: docker push

The Docker image is pushed to a container registry (AWS ECR, Harbor, DockerHub). Each push creates a new versioned image. The registry stores all versions.

Commands:

`docker login registry.techcorp.io`

`docker push registry.techcorp.io/payment-service:47`

Now any Kubernetes cluster can pull this exact image to run.

Output: image stored in registry, accessible from any K8s cluster

Stage 12: Deploy to Kubernetes (Helm)

Deployment

Action: helm upgrade --install

Jenkins runs Helm to deploy the new image to a Kubernetes cluster:

`helm upgrade --install payment-service ./helm/payment-service \`

`--set image.tag=47 --namespace production`

Helm reads the chart, fills in template values, and applies the resulting YAML to Kubernetes. K8s pulls the new image, creates new pods, performs a rolling update (replaces old pods with new ones gradually).

Output: new version of the app is running in Kubernetes pods

Stage 13: Blue-Green / Smoke Tests

Verification

Action: curl health endpoints

After deployment, the pipeline runs smoke tests against the new version. If blue-green deployment is used, the new version (green) runs alongside the old (blue) but doesn't receive traffic yet. Smoke tests hit the green endpoints. If they pass, traffic is switched from blue to green at the load balancer. If they fail, traffic stays on blue and green is destroyed.

Output: production traffic now served by new version, or rollback if smoke tests failed

Stage 14: Post-Deployment Notifications

Notification

Action: Slack/Email

Jenkins sends notifications to teams about pipeline status:

- success: green message to #deployments channel
- failure: red alert + page on-call engineer
- The Jenkinsfile post { } block handles this

Output: team knows what happened. No silent failures.

Building It: Infrastructure and Hands-On Setup

The practical implementation

8.1 Infrastructure Plan

For a learning project, use AWS EC2 with multiple instances or one larger instance. Production enterprises use multiple dedicated servers + Kubernetes cluster.

Server	Specs	Purpose	Port
Jenkins	t3.medium (2 vCPU, 4GB)	CI/CD orchestrator	8080
SonarQube	t3.medium (2 vCPU, 4GB) + PostgreSQL	Code quality analysis	9000
Nexus	t3.medium (2 vCPU, 4GB)	Artifact repository	8081
Tomcat (optional)	t3.small (1 vCPU, 2GB)	WAR deployment target	8082
Kubernetes	t3.large (2 vCPU, 8GB) + Minikube	Container orchestration	6443

8.2 Step-by-Step Setup Sequence

Follow this order. Each step depends on the previous.

Step 1: Launch EC2 instances (Ubuntu 22.04 LTS) with security group ports 22, 8080, 8081, 8082, 9000 open

Step 2: SSH into each instance, install Java 17: `sudo apt install -y openjdk-17-jdk`

Step 3: Install Jenkins on Jenkins server: see Jenkins install commands below

Step 4: Install Maven on Jenkins server: `sudo apt install -y maven`

Step 5: Install Docker on Jenkins server: `sudo apt install -y docker.io`

Step 6: Install SonarQube on SonarQube server (with PostgreSQL)

Step 7: Install Nexus on Nexus server

Step 8: Install Tomcat 9 (if using WAR deployment) on Tomcat server

Step 9: Set up Minikube + Helm on Kubernetes server (if using K8s)

Step 10: Push your Java project to GitHub

Step 11: Configure Jenkins job pointing at GitHub repo

Step 12: Add Jenkinsfile to repo with all stages

Step 13: Configure GitHub webhook -> Jenkins

Step 14: Push code, watch pipeline run end-to-end

8.3 Key Installation Commands

```
# Jenkins installation sudo wget -O /usr/share/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key echo "deb
[signed-by=/usr/share/keyrings/jenkins-keyring.asc] \ https://pkg.jenkins.io/debian-stable
binary/" | \ sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null sudo apt update && sudo
apt install -y jenkins sudo systemctl enable --now jenkins sudo cat
/var/lib/jenkins/secrets/initialAdminPassword # SonarQube installation (after Java + PostgreSQL
ready) cd /opt && sudo wget https://binaries.sonarsource.com/.../sonarqube-10.4.zip sudo unzip
sonarqube-10.4.zip && sudo mv sonarqube-10.4 sonarqube sudo adduser sonar --no-create-home
--disabled-password --gecos "" sudo chown -R sonar:sonar /opt/sonarqube sudo -u sonar
/opt/sonarqube/bin/linux-x86-64/sonar.sh start # Nexus installation cd /opt && sudo wget
https://download.sonatype.com/nexus/3/nexus.tar.gz sudo tar -xvzf nexus.tar.gz && sudo mv
```

```
nexus-* nexus sudo adduser nexus --no-create-home --disabled-password --gecos "" sudo chown -R
nexus:nexus /opt/nexus /opt/sonatype-work sudo -u nexus /opt/nexus/bin/nexus start
```

8.4 The Complete Production Jenkinsfile

This is what an enterprise Java CI/CD Jenkinsfile actually looks like:

```
pipeline { agent any tools { maven 'Maven-3.9' jdk 'JDK-17' } environment { APP_NAME =
'payment-service' IMAGE_NAME = "registry.techcorp.io/${APP_NAME}" IMAGE_TAG =
"${env.BUILD_NUMBER}" } stages { stage('Checkout') { steps { checkout scm } } stage('Build') {
steps { sh 'mvn clean package -DskipTests' } } stage('Unit Tests') { steps { sh 'mvn test' }
post { always { junit 'target/surefire-reports/*.xml' } } } stage('SonarQube Analysis') { steps
{ withSonarQubeEnv('SonarQube') { sh "mvn sonar:sonar -Dsonar.projectKey=${APP_NAME}" } } }
stage('Quality Gate') { steps { timeout(time: 5, unit: 'MINUTES') { waitForQualityGate
abortPipeline: true } } } stage('OWASP Dependency Check') { steps { sh 'mvn
dependency-check:check -DfailBuildOnCVSS=7' } } stage('Push to Nexus') { steps { sh 'mvn deploy
-DskipTests' } } stage('Docker Build & Push') { steps { sh "docker build -t
${IMAGE_NAME}:${IMAGE_TAG} ." withCredentials([usernamePassword( credentialsId:
'registry-creds', usernameVariable: 'U', passwordVariable: 'P')]) { sh "echo $P | docker login
-u $U --password-stdin" sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}" } } } stage('Deploy to
Kubernetes') { when { branch 'main' } steps { sh "" helm upgrade --install ${APP_NAME} ./helm
\ --set image.tag=${IMAGE_TAG} \ --namespace production "" } } } post { success { slackSend
channel: '#deploy', message: "OK ${APP_NAME} #${BUILD_NUMBER}" } failure { slackSend channel:
'#deploy', message: "FAIL ${APP_NAME} #${BUILD_NUMBER}" } always { cleanWs() } } }
```

Jenkins Shared Libraries (Enterprise Pattern)

How large organizations enforce standards

In an enterprise with 50+ microservices, you do NOT want 50 copies of the Jenkinsfile above. You want one centrally-controlled definition that all teams use. That is what Shared Libraries solve.

9.1 The Problem

Without Shared Libraries:

- 50 microservices = 50 Jenkinsfiles
- Security team wants to add new scan: 50 PRs across 50 repos
- Some teams forget to update -> security scan missing in 3 services
- Different teams use slightly different syntax for the same stage
- Audit finds inconsistencies, compliance risk

9.2 The Solution

A separate Git repository called "jenkins-shared-library" containing reusable pipeline functions. Every Jenkinsfile imports it and calls library functions instead of writing logic directly.

```
jenkins-shared-library/ |-- vars/ # callable functions live here | |-- javaPipeline.groovy #
def call() = entire pipeline | |-- runSecurityScans.groovy # def call() = SonarQube + OWASP |
`-- dockerBuildPush.groovy # def call() = build + push |-- src/ # Groovy classes for helpers |
`-- com/techcorp/Utils.groovy `-- resources/ # static config files
```

A developer's Jenkinsfile then becomes 3 lines:

```
@Library('jenkins-shared-library@v2.1.0') _ javaPipeline(appName: 'payment-service')
```

9.3 Security Enforcement Through Shared Libraries

This is the key enterprise benefit. The shared library repo has admin-only write access. Developers have read-only access. Inside javaPipeline.groovy, the security scan stages are mandatory:

```
// vars/javaPipeline.groovy (admin-controlled) def call(Map config = [:]) { pipeline { agent
any stages { stage('Build') { steps { sh 'mvn clean package' } } stage('Tests') { steps { sh
'mvn test' } } stage('Sonar') { steps { runSonarQube() } } // CANNOT SKIP stage('OWASP') {
steps { sh 'mvn dependency-check:check' } } // CANNOT SKIP stage('Deploy') { steps { /* ... */
} } } }
```

Developers call javaPipeline() but cannot modify it. Security stages always run.

Benefit	Detail
Code reuse	50 services use the same pipeline. Update once, applies everywhere.
Security enforcement	Developers cannot disable security scans - they don't control the library.
Compliance audit	One library to audit instead of 50 Jenkinsfiles.
Standardization	Every pipeline runs the same stages in the same order.
Reduced expertise	Dev teams write 3-line Jenkinsfiles, no need to be Jenkins experts.
Versioned	@Library('lib@v2.1.0') - pin to stable versions, upgrade when ready.

Java vs Python vs Node.js: Deep Comparison

Same pipeline shape, fundamentally different tools

The pipeline SHAPE is identical across all three languages: Checkout -> Dependencies -> Build -> Test -> Quality -> Security -> Artifact -> Docker -> Deploy. What changes is the tools and the philosophy. Understanding these differences is what separates a true DevOps engineer from a tutorial-follower.

10.1 The Fundamental Difference: Compilation Philosophy

Aspect	Java	Python	Node.js
Language type	Compiled to bytecode	Interpreted	Interpreted (TS = compiled)
Build step	MANDATORY (mvn compile)	NOT needed	Optional (npm build for frontend)
Runtime	JVM executes bytecode	Python interpreter reads source	V8 engine reads source
Errors caught	At compile time	At runtime	At runtime (TS at compile)
Typing	Strong, static	Dynamic, optional hints	Dynamic (TS = static)
Artifact	.jar / .war (binary)	Source code or .whl	Source code or bundled JS

10.2 Why Java Is Built This Way

Java's design (compiled, strong-typed, JVM-based) was deliberate and shapes everything:

- Enterprise focus:** Java was designed for large teams building mission-critical systems. Strict typing catches errors before code reaches production. A typo that crashes Python at 2am is caught by Java compiler at 2pm.
- Performance:** JVM bytecode runs much faster than interpreted languages. JIT (Just-In-Time) compilation makes hot code paths run at near-native speed. For a banking system processing 10,000 transactions per second, this matters.
- Platform independence:** "Write once, run anywhere" was revolutionary. Same WAR file runs on Linux, Windows, AIX, Solaris — whatever the bank's legacy systems use.
- Ecosystem maturity:** 30 years of development -> Maven, Spring, Hibernate, Tomcat — battle-tested libraries that handle every enterprise problem you can imagine.
- Backwards compatibility:** Java code from 2005 still compiles and runs today. This matters when a bank has 20-year-old code that handles customer accounts.

KEY INSIGHT: Python and Node.js prioritize developer productivity (fast iteration, less ceremony). Java prioritizes enterprise reliability (catch errors early, scale to massive teams, run for decades). Neither is "better" — they're optimized for different problems.

10.3 Stage-by-Stage Pipeline Comparison

Stage	Java	Python	Node.js
Dependencies	mvn (reads pom.xml) Downloads from Maven Central	pip install -r requirements.txt Downloads from PyPI	npm install (reads package.json) Downloads from npmjs.com
Build	mvn clean package Compiles .java -> .class -> .war	(no build, source IS the artifact)	npm run build (frontend) Not needed for backend Node
Test	mvn test (JUnit) Results: target/surefire-reports/	pytest --junitxml=results.xml	npm test (Jest, Mocha) or jest --reporters=jest-junit
Coverage	JaCoCo (mvn verify) XML coverage report	pytest-cov coverage.xml	Istanbul / nyc coverage/lcov.info
SAST	mvn sonar:sonar Maven plugin invokes SonarQube	sonar-scanner (standalone) + flake8, pylint, bandit	sonar-scanner (standalone) + eslint, npm audit
SCA	OWASP Dependency Check mvn dependency-check:check	safety check -r requirements.txt + optional OWASP	npm audit or snyk test
Artifact	.war / .jar -> Nexus Versioned in maven-releases	.whl -> Nexus PyPI repo OR skip to Docker	Either bundled .zip OR skip to Docker
Docker	Multi-stage: builder + JRE ~200MB final image	Single stage: python:slim + pip install ~150MB	Single stage: node:alpine + npm install --production ~100MB
Runtime	java -jar app.jar or Tomcat for WAR	gunicorn / uvicorn (production WSGI/ASGI)	node server.js or pm2 process manager

10.4 The Artifact Difference (Critical to Understand)

This is the question interviewers love to ask: "Why does Java need Nexus but Python often doesn't?"

Java pipeline: Code -> Compile -> Binary .war/.jar -> Nexus -> Docker -> K8s

The .war file is the deliverable. It is a binary artifact that needs versioned storage. Nexus stores it. Multiple environments can pull the same .war and deploy it.

Python/Node.js pipeline: Code -> (no compile) -> Docker -> Registry -> K8s

There is no separate binary to store. The "artifact" IS the Docker image. The image registry (ECR, DockerHub) plays the role Nexus plays for Java. So storing a .whl or .zip is redundant when you're already building a Docker image.

In modern containerized environments, even Java pipelines sometimes skip Nexus and go straight to Docker — the Docker image is the only artifact needed. But traditional enterprise Java keeps Nexus because:

- Some apps still deploy WAR to Tomcat (not Docker)
- Internal Java libraries need a Maven repo for other projects to depend on
- Compliance audits prefer the immutable .war versioning Nexus provides

Interview Preparation: 70+ Questions with Answers

Practice these out loud, not just by reading

Questions are grouped by topic. Practice answering each in 2-3 sentences out loud. Reading is not preparing. Saying the answer out loud is.

Java & JVM Fundamentals

Q1. What is the difference between JDK, JRE, and JVM?

JVM (Java Virtual Machine) executes bytecode. JRE (Java Runtime Environment) is JVM plus standard libraries — what you need to RUN Java apps. JDK (Java Development Kit) is JRE plus development tools like javac compiler — what you need to BUILD Java apps. CI servers need JDK. End-user machines only need JRE.

Q2. What is Java bytecode?

Bytecode is the platform-independent intermediate format produced when javac compiles .java source files. It is binary data with file extension .class. The JVM reads bytecode and translates it to machine code at runtime. This is what enables "Write Once, Run Anywhere".

Q3. Why is Java a compiled language but called platform-independent?

Java compiles to bytecode, not native machine code. Bytecode is platform-independent. The JVM, which is platform-specific, translates bytecode to machine code at runtime. So Java code is compiled once and runs on any platform that has a JVM.

Q4. What is an LTS Java version and why does it matter?

LTS = Long Term Support. Oracle releases LTS versions every 2-3 years and supports them for many years. Java 8, 11, 17, 21 are LTS. Non-LTS versions (12-16, 18-20) are short-lived and not used in production. Enterprises run LTS versions for stability and security patches.

Maven

Q1. What is Maven?

Maven is a build automation tool for Java projects. It reads pom.xml to manage dependencies, compile source code, run tests, and package the result into JAR/WAR files. It standardizes the build process so any developer can build any Maven project the same way.

Q2. What is pom.xml?

POM = Project Object Model. pom.xml is the configuration file for a Maven project. It defines the project's coordinates (groupId, artifactId, version), dependencies, plugins, and build settings. Every Maven project has one pom.xml at its root.

Q3. Explain Maven coordinates: groupId, artifactId, version.

These three values uniquely identify a Maven artifact. groupId is the organization namespace (com.company). artifactId is the specific project name (payment-service). version is the version number (1.0.5). Together they form the artifact's "address" in any Maven repository.

Q4. What does mvn clean package do?

It runs two Maven goals: "clean" deletes the target/ folder (removes previous build output), then "package" executes the lifecycle: validate -> compile (.java to .class) -> test (runs JUnit) -> package (bundles into .war or .jar). The result is a deployable artifact in target/.

Q5. What is the difference between mvn package and mvn deploy?

mvn package creates the artifact in target/ locally. mvn deploy does everything package does PLUS uploads the artifact to the remote repository defined in pom.xml distributionManagement (typically Nexus). Used in CI/CD to push every successful build to Nexus.

Q6. Explain Maven dependency scopes.

compile (default): needed at compile time AND bundled in artifact. provided: needed at compile time but NOT bundled (runtime environment provides it, e.g., servlet-api with Tomcat). runtime: only needed at runtime (e.g., JDBC drivers). test: only needed for tests (JUnit). system: like provided but with manual JAR path.

Q7. What does provided mean and what happens if you remove it?

It means the library is needed for compilation but should NOT be packaged in the WAR — the runtime (Tomcat) provides it. If removed (default is compile), the servlet-api.jar would be bundled inside your WAR. Tomcat also has it, so two copies exist, causing ClassLoader conflicts and runtime errors.

Q8. What is the Maven build lifecycle?

A predefined sequence of phases: validate, compile, test, package, verify, install, deploy. Each phase includes all previous phases. mvn deploy runs everything from validate through deploy. You cannot skip phases.

Q9. What is the difference between mvn install and mvn deploy?

mvn install copies the artifact to your LOCAL Maven repository at ~/.m2/repository/, making it available to other projects on your machine. mvn deploy uploads it to a REMOTE repository (Nexus, Artifactory), making it available to the entire team.

Q10. What is Maven Central?

The default public Maven repository. Contains millions of open-source Java libraries. When Maven sees a dependency it doesn't have locally, it downloads from Maven Central by default. Enterprises proxy Maven Central through Nexus to control access and cache downloads.

Java Web Applications

Q1. What is a .war file?

WAR = Web Application aRchive. A zip-format file containing compiled .class files, dependency JARs (those not provided-scope), WEB-INF/web.xml or annotations, and static resources (HTML, CSS, JS). It requires a servlet container like Tomcat to run.

Q2. What is the difference between JAR, WAR, and EAR?

JAR (Java ARchive): standalone Java apps or libraries. WAR (Web ARchive): web applications, deployed in servlet containers like Tomcat. EAR (Enterprise ARchive): enterprise apps with multiple modules, deployed in full Jakarta EE servers like WildFly. EAR is rare in modern projects.

Q3. What is Tomcat?

Tomcat is a servlet container - a runtime that provides HTTP listening, request routing, and servlet lifecycle management for Java web apps. It listens on a port (default 8080), reads .war files from webapps/, and routes incoming HTTP requests to the appropriate servlet.

Q4. What is the Tomcat webapps/ folder?

The deployment folder. When you copy a .war file into webapps/, Tomcat automatically detects it, extracts it into a folder, and deploys the application — no restart needed. This is called hot deployment.

Q5. Where do you look when a Tomcat deployment fails?

/opt/tomcat9/logs/catalina.out — Tomcat's main log file. It contains deployment events, ClassNotFoundExceptions, port conflicts, and startup errors. Use "tail -f /opt/tomcat9/logs/catalina.out" to watch live.

Jenkins & Pipelines

Q1. What is Jenkins?

Jenkins is an open-source automation server that runs CI/CD pipelines. It reads a Jenkinsfile from your Git repository, executes stages on build agents, and records build history with logs and artifacts. Runs as a system service on port 8080.

Q2. What is a Jenkinsfile?

A text file named "Jenkinsfile" at the root of your Git repo defining the pipeline as code in Groovy DSL syntax. Because it lives in the repo, pipeline changes are version-controlled and reviewable through pull requests.

Q3. What is the difference between a Jenkins Freestyle job and a Pipeline job?

Freestyle: configured through Jenkins UI, not version-controlled, limited to linear steps. Legacy approach. Pipeline: defined in Jenkinsfile in Git, version-controlled, supports complex multi-stage flows, parallel execution, and conditionals. Modern enterprise standard.

Q4. What is "agent any" in a Jenkinsfile?

It means: run this pipeline on any available Jenkins agent. Jenkins picks the first free one. In production, you would use "agent { label 'linux' }" or "agent { docker { image 'maven:3.9' } }" to control exactly where builds run.

Q5. What is the Jenkins workspace?

A folder on the Jenkins agent at `/var/lib/jenkins/workspace/` where Jenkins clones the repository and runs all build commands. Every build starts with a fresh clone — intentional, to prevent stale-state bugs.

Q6. How does Jenkins manage credentials securely?

Jenkins encrypts credentials in `/var/lib/jenkins/secrets/`. Add them in Manage Jenkins -> Credentials. Reference them by ID in Jenkinsfile using `withCredentials()` — the password never appears in console logs because Jenkins automatically masks it.

Q7. What is the `post { }` block in a Jenkinsfile?

Actions that run AFTER all stages, regardless of outcome. `always{ }` runs no matter what. `success{ }` only if all stages passed. `failure{ }` if any stage failed. Used for notifications, workspace cleanup, reporting.

Q8. How do you run stages in parallel in Jenkins?

Use a parallel block: `stage('Parallel Tests') { parallel { stage('Unit'){...} stage('Integration'){...} } }`. Both stages run simultaneously. Pipeline waits until both complete before continuing. Reduces total build time.

Q9. What is a Jenkins agent?

A worker machine (or Docker container, or Kubernetes pod) that runs build commands. The Jenkins controller (master) orchestrates but does not run builds — it delegates to agents for security and performance reasons.

Q10. What is a Jenkins Shared Library?

A separate Git repository containing reusable Groovy functions that any Jenkinsfile can import. Standard structure: `vars/` for callable functions, `src/` for classes, `resources/` for static files. Allows DRY pipelines across many repositories and enforces security standards admin-only.

Q11. How does a GitHub webhook trigger a Jenkins build?

When code is pushed to GitHub, GitHub sends an HTTP POST to Jenkins at /github-webhook/. Jenkins receives the webhook, identifies the matching job by repository URL, and queues a build. Configured in GitHub: Settings -> Webhooks -> Add webhook -> Payload URL: http://:8080/github-webhook/

CI/CD Concepts

Q1. What is the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment?

CI: every push triggers automated build and tests. Continuous Delivery: every successful build is auto-prepared for release but requires manual approval to actually deploy. Continuous Deployment: every successful build deploys to production automatically. Enterprises typically use CI + Continuous Delivery.

Q2. Walk me through your CI/CD pipeline.

Developer pushes to GitHub. Jenkins webhook triggers a build agent. Agent clones code, runs mvn clean package, executes JUnit tests, sends code to SonarQube for SAST analysis (Quality Gate must pass), runs OWASP Dependency Check for SCA, uploads .war to Nexus, builds and pushes Docker image, then Helm deploys to Kubernetes with rolling update. Notifications sent to Slack on completion.

Q3. What happens when a stage fails in your pipeline?

The pipeline stops at the failed stage. Subsequent stages are skipped. Jenkins marks the build as FAILED. The team is notified. No artifact is created, no deployment happens. The developer fixes the issue, pushes again, and a new build is triggered.

Q4. What is Pipeline as Code and why is it valuable?

Pipeline as Code means the pipeline definition lives in the source repository (Jenkinsfile). Benefits: version controlled, reviewable through PRs, reproducible, auditable. The alternative — UI-configured pipelines — is fragile and impossible to audit. Every modern enterprise uses Pipeline as Code.

Q5. Why store artifacts in Nexus instead of rebuilding every time?

Reproducibility: same artifact tested in QA is the exact binary deployed to production. Speed: downstream stages pull from Nexus instead of rebuilding. Rollback: any previous version immediately available without rebuild. Audit: complete history of every version built.

Security & DevSecOps

Q1. What is SAST?

Static Application Security Testing. Tools like SonarQube analyze source code WITHOUT running it to find bugs, vulnerabilities, code smells. Runs in the CI pipeline before artifact creation. Catches issues like SQL injection patterns, hardcoded passwords, null pointer dereferences.

Q2. What is SCA?

Software Composition Analysis. Tools like OWASP Dependency Check or NexusIQ analyze your third-party dependencies (libraries) for known CVEs in the National Vulnerability Database. Catches things like Log4Shell (CVE-2021-44228) in dependencies you import.

Q3. What is the difference between SAST and SCA?

SAST scans YOUR source code for issues. SCA scans the LIBRARIES you import (third-party code) for known security vulnerabilities. Your code might be safe (SAST passes) but vulnerable through a library (SCA catches it). Both are needed.

Q4. What is a CVE?

Common Vulnerabilities and Exposures. A unique ID for a publicly disclosed security vulnerability. Example: CVE-2021-44228 = Log4Shell. Maintained in the National Vulnerability Database (NVD). SCA tools compare your dependencies against this database.

Q5. What is a CVSS score?

Common Vulnerability Scoring System. A 0-10 severity score for a CVE. 0-3.9 Low, 4-6.9 Medium, 7-8.9 High, 9-10 Critical. Enterprise policy typically blocks builds with Critical or High CVEs. Configure OWASP with `-DfailBuildOnCVSS=7`.

Q6. What is a SonarQube Quality Gate?

A pass/fail policy applied to code metrics. Example: "no new bugs, no new vulnerabilities, coverage >= 80%, duplication < 3%". If the Quality Gate fails, Jenkins aborts the pipeline. This is how security/quality standards are enforced automatically.

Q7. A legacy codebase has 500 SonarQube violations. How do you handle the Quality Gate?

Don't try to fix 500 issues before going live. Set the Quality Gate to "new code only" mode in SonarQube. The 500 existing issues become tracked technical debt. New code must be clean. This is the standard pragmatic enterprise approach.

Docker

Q1. What is the difference between a container and a VM?

VM virtualizes the entire hardware — each VM has its own OS (heavy, slow, GB-sized). Container virtualizes only the application — shares the host OS kernel (light, fast, MB-sized). Containers start in seconds; VMs take minutes.

Q2. What is the difference between an image and a container?

An image is a read-only blueprint — a packaged application + dependencies. A container is a running instance of an image. Same relationship as a class and an object in OOP. You build an image once, then run many containers from it.

Q3. Walk me through a Dockerfile for a Java app.

FROM base image (maven for build, jre for run). WORKDIR sets the working directory. COPY copies files from host to image. RUN executes commands during build. EXPOSE documents the port. ENTRYPOINT sets the main executable. CMD sets default arguments. Multi-stage build separates build tools from runtime image.

Q4. What is a multi-stage Docker build?

A Dockerfile with multiple FROM statements. Stage 1 (builder) uses a heavy image with build tools (Maven, JDK) to compile. Stage 2 (runtime) uses a lightweight image (JRE only) and copies only the compiled artifact from stage 1. Result: smaller, more secure final image.

Q5. What is Docker layer caching?

Each Dockerfile instruction creates a read-only layer. Docker caches layers — if an instruction and its inputs haven't changed, Docker reuses the cache. Best practice: order Dockerfile from most-stable (base image, dependencies) to least-stable (source code) to maximize cache hits.

Q6. What is the difference between COPY and ADD in Dockerfile?

COPY: copies files from host to image. Simple, explicit. ADD: does everything COPY does PLUS can extract tar files and download from URLs. Best practice: always use COPY unless you specifically need ADD's extra features.

Q7. What is the difference between ENTRYPOINT and CMD?

ENTRYPOINT: the executable that always runs. Cannot be easily overridden. CMD: default arguments to ENTRYPOINT or the default command. Can be overridden at "docker run". Together: ENTRYPOINT ["java", "-jar"] CMD ["app.jar"] makes the container "java -jar app.jar" by default.

Q8. What is the difference between docker run and docker exec?

docker run creates a NEW container from an image and starts it. docker exec runs a command inside an ALREADY RUNNING container. Use exec to debug a running container: "docker exec -it container-id bash" opens a shell inside it.

Kubernetes

Q1. What is Kubernetes?

A container orchestration platform. It runs containers across many machines, handles failures, scales them based on load, manages networking between them. Docker runs containers on one machine; Kubernetes runs them across hundreds of machines as a single system.

Q2. What is a Pod?

The smallest deployable unit in Kubernetes. A Pod wraps one (sometimes multiple) container with shared storage, network, and configuration. In practice, most Pods have a single container — the Pod is the unit of scheduling, the container is the workload.

Q3. What is a Deployment in Kubernetes?

A higher-level object that manages multiple identical Pods. Defines desired state (e.g., "run 3 replicas of my-app version 1.0.5"). Kubernetes constantly reconciles actual state with desired state — if a Pod crashes, it creates a new one.

Q4. What is a Service in Kubernetes?

A stable network endpoint to access Pods. Pods come and go; their IPs change. A Service provides a consistent DNS name and IP that routes to whatever Pods match its selector. Service types: ClusterIP (internal), NodePort (external via node IP), LoadBalancer (cloud load balancer).

Q5. How do you debug a Pod in CrashLoopBackOff state?

1. `kubectl get pods` (confirm state). 2. `kubectl describe pod` (events, error reasons). 3. `kubectl logs` (application logs). 4. `kubectl logs --previous` (logs from previous crash). 5. If image issue: check `kubectl describe events`. If config issue: check ConfigMap/Secret mounts.

Q6. What is Helm?

A package manager for Kubernetes. Bundles related Kubernetes YAML files (Deployment, Service, ConfigMap, etc.) into a reusable "chart". Templates them with placeholder values. Allows deploying with one command: `helm upgrade --install my-app ./chart --set image.tag=1.0.5`.

Q7. What is blue-green deployment?

Two identical production environments. Blue is live serving users. Deploy the new version to Green. Test it. Switch the load balancer from Blue to Green — zero downtime. If Green fails, switch back to Blue. Instant rollback. Easy to implement in Kubernetes via Service selector changes.

Q8. What is the difference between a Deployment and a StatefulSet?

Deployment: for stateless applications (web servers, APIs). Pods are interchangeable, can be replaced freely. StatefulSet: for stateful applications (databases). Pods have stable identities (pod-0, pod-1), ordered startup/shutdown, persistent storage tied to each Pod.

AWS Services

Q1. What is EC2?

Elastic Compute Cloud. AWS virtual servers (instances). You choose the OS, CPU, RAM, storage. Pay per hour of running time. Used to host Jenkins, SonarQube, Nexus, application servers in this pipeline.

Q2. What is the difference between stopping and terminating an EC2 instance?

Stop: instance halts, compute billing stops, EBS volume preserved, data safe. You can start it again. Public IP changes. Terminate: instance permanently deleted. EBS root volume deleted by default. Irreversible. All data lost.

Q3. What is an IAM Role and why use it instead of access keys on EC2?

An IAM Role grants an EC2 instance permissions to call AWS services (S3, ECR) without storing access keys. AWS rotates credentials automatically. If the instance is compromised, no keys are stolen. Always prefer IAM Roles over hardcoded access keys.

Q4. What is a Security Group?

A virtual firewall at the instance level. Stateful — return traffic is automatically allowed if you permit inbound. Applied to specific instances. Different from Network ACLs which work at subnet level and are stateless.

Q5. What is S3?

Simple Storage Service. Object storage in AWS. Unlimited storage in buckets, accessed via API or URLs. 11 nines durability. Used in CI/CD for: artifact backups, log archiving, static website hosting, Terraform state, build outputs.

Q6. What is the difference between ALB and NLB?

ALB (Application Load Balancer): Layer 7, HTTP/HTTPS, can route based on URL path, host headers. Terminates SSL. NLB (Network Load Balancer): Layer 4, TCP/UDP, ultra-low latency, static IP, used for non-HTTP protocols. For web apps and DevOps pipelines: always ALB.

Q7. What is an Auto Scaling Group?

A group of EC2 instances managed as one unit. Automatically adds or removes instances based on scaling policies (CPU, custom metrics, schedule). Defined by Launch Template + min/max/desired capacity. Replaces unhealthy instances automatically.

Q8. How do ASG and ALB work together?

ASG manages the pool of instances (adding/removing). ALB distributes traffic across them. When ASG adds a new instance, it registers with the ALB target group. When ASG removes one, it deregisters from ALB first (connection draining) then terminates. Together: automatic scaling + automatic traffic management.

Git & Gitflow

Q1. What is the difference between git merge and git rebase?

merge creates a new merge commit combining two branches, preserves full history of both. rebase rewrites history by replaying commits on top of another branch — creates linear history. Rule: rebase locally for cleanup, never rebase shared branches because it conflicts with others' copies.

Q2. What is the difference between git fetch and git pull?

fetch downloads changes from remote but does NOT update your working branch. pull = fetch + merge in one command, updates your local branch immediately. Best practice: fetch first, inspect with "git diff origin/main", then merge consciously.

Q3. How do you undo a commit that has been pushed to a shared branch?

Use "git revert ", not "git reset". Reset rewrites history — dangerous on shared branches. Revert creates a NEW commit that undoes the changes. History is preserved. Safe for shared branches.

Q4. What is Gitflow?

A branching strategy with specific branch roles: main (production only, tagged releases), develop (integration), feature/* (new features off develop), release/* (stabilization off develop), hotfix/* (emergency fixes off main). Features merge to develop; releases merge to main AND develop.

Q5. What is git cherry-pick?

Applies a specific commit from one branch to another without merging the entire branch. Use case: a bugfix from develop needs to go to release/1.0.1 without bringing other develop commits. "git cherry-pick ".

Q6. What is the difference between git reset --soft, --mixed, and --hard?

All move HEAD back to a previous commit. --soft: changes stay STAGED, ready to recommit. --mixed (default): changes go to UNSTAGED, need git add again. --hard: changes are DELETED, working dir reset. --hard is dangerous - you lose work.

Q7. What is git stash?

Temporarily saves uncommitted changes so you can switch branches without committing incomplete work. "git stash" saves and reverts working directory. "git stash pop" restores them later. Useful when urgent context switching is needed mid-feature.

SECTION 12

Commands Cheat Sheet

Every command you need, organized by tool

MAVEN

```
# Lifecycle commands mvn clean # delete target/ folder mvn clean compile # compile
.java -> .class only mvn clean test # compile + run JUnit tests mvn clean package #
compile + test + create WAR/JAR mvn clean install # everything + copy to ~/.m2/ mvn
clean deploy # everything + push to Nexus mvn package -DskipTests # skip tests
(faster but unsafe) # Inspection mvn -version # check installed version mvn
dependency:tree # show all transitive dependencies mvn dependency:analyze # find
unused/missing deps mvn versions:display-dependency-updates # newer versions
available # SonarQube & security mvn sonar:sonar # send analysis to SonarQube mvn
dependency-check:check # OWASP scan
```

JENKINS

```
# Service management sudo systemctl start jenkins sudo systemctl stop jenkins sudo
systemctl restart jenkins sudo systemctl status jenkins sudo systemctl enable jenkins
# auto-start on boot # First-time setup sudo cat
/var/lib/jenkins/secrets/initialAdminPassword # Logs tail -f
/var/log/jenkins/jenkins.log # Useful directories /var/lib/jenkins/ # Jenkins home
/var/lib/jenkins/workspace/ # job workspaces /var/lib/jenkins/jobs/ # job
configurations /var/lib/jenkins/secrets/ # credentials store
```

TOMCAT

```
# Lifecycle sudo /opt/tomcat9/bin/startup.sh sudo /opt/tomcat9/bin/shutdown.sh #
Deploy a WAR sudo cp app.war /opt/tomcat9/webapps/ # Remove a deployment sudo rm -rf
/opt/tomcat9/webapps/old-app* # Debug tail -f /opt/tomcat9/logs/catalina.out ls
/opt/tomcat9/webapps/ # see deployed apps
```

SONARQUBE

```
# Service sudo -u sonar /opt/sonarqube/bin/linux-x86-64/sonar.sh start sudo -u sonar
/opt/sonarqube/bin/linux-x86-64/sonar.sh stop sudo -u sonar
/opt/sonarqube/bin/linux-x86-64/sonar.sh status # Standalone scanner (non-Maven
projects) sonar-scanner \ -Dsonar.projectKey=my-app \ -Dsonar.sources=src \
-Dsonar.host.url=http://sonar:9000 # UI http://:9000 (admin/admin first time)
```

NEXUS

```
# Lifecycle sudo -u nexus /opt/nexus/bin/nexus start sudo -u nexus
/opt/nexus/bin/nexus stop sudo -u nexus /opt/nexus/bin/nexus status # First-time
password cat /opt/sonatype-work/nexus3/admin.password # Logs tail -f
/opt/sonatype-work/nexus3/log/nexus.log # UI http://:8081 Repos: maven-releases,
maven-snapshots, maven-public
```

DOCKER

```
# Images docker build -t myapp:1.0 . docker images docker pull ubuntu:22.04 docker
push myrepo/myapp:1.0 docker rmi myapp:1.0 # Containers docker run -d -p 8080:8080
--name app myapp:1.0 docker ps # running docker ps -a # all docker logs app docker
logs -f app # follow docker exec -it app bash docker stop app docker rm app # Cleanup
docker system prune -a # remove everything unused
```

KUBERNETES

```
# Inspect kubectl get pods kubectl get deployments kubectl get services kubectl get
nodes kubectl describe pod kubectl logs kubectl logs --previous # crashed pod kubectl
exec -it -- bash # Apply manifests kubectl apply -f deployment.yaml kubectl delete -f
deployment.yaml # Rollout kubectl rollout status deployment/ kubectl rollout undo
deployment/ kubectl rollout history deployment/
```

HELM

```
# Install / upgrade helm install my-app ./chart helm upgrade my-app ./chart helm
upgrade --install my-app ./chart # idempotent # Inspect helm list helm status my-app
helm history my-app # Rollback helm rollback my-app 1 # to revision 1 # Uninstall
helm uninstall my-app
```

GIT

```
# Daily workflow git status git add . git commit -m "message" git push origin git
pull origin git fetch origin # Branches git checkout -b feature/login git switch
feature/login git merge feature/login git branch -d feature/login # Inspect & undo
git log --oneline --graph git diff git revert git stash / git stash pop
```

AWS CLI

```
# Configure aws configure # EC2 aws ec2 describe-instances aws ec2 start-instances
--instance-ids i-xxx aws ec2 stop-instances --instance-ids i-xxx # S3 aws s3 cp
file.war s3://bucket/path/ aws s3 sync ./dist s3://bucket/ aws s3 ls s3://bucket/ #
ECR (container registry) aws ecr get-login-password | docker login --username AWS
--password-stdin .dkr.ecr.region.amazonaws.com aws ecr describe-repositories
```

Glossary: Every Term Defined

Quick lookup for every concept

Agent (Jenkins) A worker machine, container, or pod that runs build commands. Controller orchestrates; agents execute.	AMI Amazon Machine Image. Template for creating EC2 instances.
Artifact The compiled output of a build. For Java: .war or .jar files. Stored in Nexus.	ASG Auto Scaling Group. AWS service that adds/removes EC2 instances based on demand.
Bytecode Platform-independent intermediate format produced by javac. Executed by JVM. The reason Java is portable.	CI Continuous Integration. Every code push triggers automated build and tests.
CD Continuous Delivery (with approval) or Continuous Deployment (fully automatic). The post-CI stages.	CVE Common Vulnerabilities and Exposures. Unique ID for a known security vulnerability.
CVSS Common Vulnerability Scoring System. Severity score 0-10 for a CVE.	Deployment (Kubernetes) A K8s object managing multiple identical Pods. Handles rolling updates and rollbacks.
EBS Elastic Block Store. AWS virtual hard disks attached to EC2 instances.	EC2 Elastic Compute Cloud. AWS virtual servers.
Gitflow A branching strategy: main, develop, feature/, release/, hotfix/ branches with defined merge rules.	Groovy JVM-based language. Jenkinsfile is written in Groovy DSL.
Helm Kubernetes package manager. Bundles K8s YAML into reusable parameterized "charts".	IAM Identity and Access Management. AWS service controlling permissions.
JAR Java ARchive. Zip file with compiled Java classes. For libraries or standalone apps.	JDK Java Development Kit. JRE + compiler + dev tools. Needed to BUILD Java.
Jenkinsfile A text file in your repo defining the CI/CD pipeline as code in Groovy DSL.	JRE Java Runtime Environment. JVM + standard libraries. Needed to RUN Java.
JUnit The standard Java unit testing framework.	JVM Java Virtual Machine. Executes bytecode. Platform-specific implementation of bytecode runtime.
LTS Long Term Support. Java versions with multi-year support (8, 11, 17, 21).	Maven Java build automation tool. Reads pom.xml to compile, test, package projects.
Maven Central The default public Maven repository. Contains millions of open-source Java libraries.	Maven Coordinates groupId + artifactId + version. Uniquely identify any Maven artifact.

Multi-stage Docker build Dockerfile with multiple FROM statements. Separates build environment from runtime image.	Nexus Artifact repository manager. Stores built .war/.jar files with version history.
OWASP Dependency Check Open-source SCA tool. Scans dependencies for known CVEs against NVD database.	Pod Smallest unit in Kubernetes. Wraps one (or more) containers with shared storage and network.
pom.xml Maven's project configuration file. Defines dependencies, plugins, build settings.	Quality Gate SonarQube pass/fail policy. Pipeline stops if Quality Gate fails.
Rolling Update Kubernetes default deployment strategy. Replaces pods one by one for zero-downtime updates.	SAST Static Application Security Testing. Scans source code without running it. SonarQube.
SCA Software Composition Analysis. Scans third-party dependencies for CVEs. OWASP, Snyk, NexusIQ.	Servlet A Java class that handles HTTP requests. Extends HttpServlet, overrides doGet/doPost.
Servlet Container Runtime for Java web apps. Tomcat, Jetty, WildFly. Provides HTTP listening and servlet lifecycle.	Service (Kubernetes) Stable network endpoint for accessing Pods. Routes traffic based on label selectors.
Shared Library (Jenkins) Separate Git repo containing reusable pipeline code. Enables DRY pipelines across many repos.	SonarQube SAST tool. Analyzes source code for bugs, vulnerabilities, code smells, coverage.
Tomcat Servlet container. Runs Java web apps. Listens on a port, serves .war files.	WAR Web Application Archive. Zip with compiled servlets + resources. Deployed in servlet containers.
Webhook HTTP callback. GitHub sends one to Jenkins when code is pushed, triggering the pipeline.	Workspace (Jenkins) Folder on Jenkins agent where code is cloned and builds run. /var/lib/jenkins/workspace/

End of Guide

You now have a complete reference for building Java CI/CD pipelines, understanding why each tool exists, and answering any interview question on the topic. Read it multiple times. Practice the interview answers out loud. Build the pipeline yourself.